

Marshmallow vs Pydantic

What is Marshmallow?

Marshmallow is a serialization, deserialization, and validation library used to convert complex Python objects into native Python data types and validate incoming data.

What is Pydantic?

Pydantic is a data validation and parsing library that uses Python type hints to validate, serialize, and deserialize data automatically.

Key Differences

Feature	Marshmallow	Pydantic
Validation Style	Schema-based	Type Hint-based
Performance	Slower	Faster
Type Safety	Limited	Strong
FastAPI Integration	Not Native	Native
Learning Curve	More Configuration	Easier
Auto Type Conversion	Limited	Excellent
API Development	Good	Excellent

Marshmallow Example

```
from marshmallow import Schema, fields

class UserSchema(Schema):
    name = fields.Str(required=True)
    age = fields.Int(required=True)
```

Pydantic Example

```
from pydantic import BaseModel

class User(BaseModel):
```

```
name: str
age: int
```

Why is Pydantic Preferred in FastAPI?

- Built-in FastAPI support
 - Automatic validation
 - Better performance
 - Uses Python type hints
 - Auto-generated OpenAPI/Swagger documentation
-

When Would You Use Marshmallow?

- Existing Flask projects
 - Legacy applications already using Marshmallow
 - Projects heavily dependent on schema-based validation
-

When Would You Use Pydantic?

- FastAPI applications
 - Modern API development
 - Type-safe applications
 - High-performance services
-

Interview Answer

"I have primarily used Pydantic with FastAPI because it provides type-hint based validation, automatic serialization/deserialization, better performance, and seamless FastAPI integration. Marshmallow follows a schema-based approach and is commonly seen in Flask ecosystems, while Pydantic is generally preferred for modern API development."